CampusFlow

Entrega 2 - Etapa intermediária

Publicação, refinamento, testes e análise técnico-ética do uso de IA

Felipe Amorim Monteiro

RA 22452139

14 de outubro de 2026

Repositório público: MorimFr/entrega-inicial

Versão candidata: PR #24; implementação homologada f863ddb.

Estado: código e relatório preparados; aprovação e publicação na main pendentes.

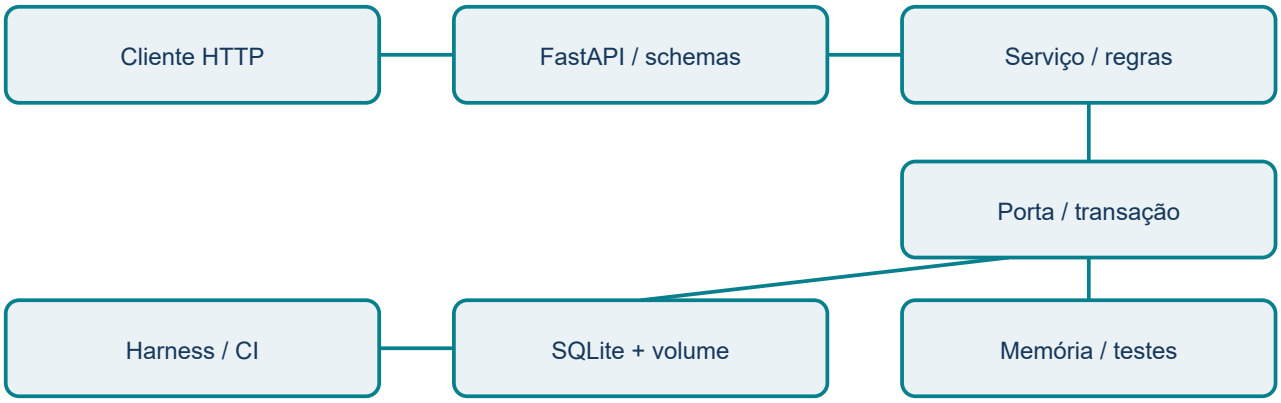
Equipe individual: autorrevisão na conta secundária, conforme autorização docente relatada pelo aluno.

1. Problema, escopo e ambiente

CampusFlow resolve conflitos no agendamento de salas de estudo. A solução é uma API integrada que lista salas, cria/consulta/cancela reservas, calcula disponibilidade e consulta reservas por usuário. A versão final persiste em SQLite e conserva regras em transações locais.

Regra	Comportamento homologado
RN-01 / RN-02	Intervalos com offset, fim posterior e duração de até 2 horas.
RN-03	Inteiro positivo estrito, limitado à capacidade da sala.
RN-04 / RN-05	Conflitos rejeitados; intervalos adjacentes permitidos.
RN-06 / RN-08	Até duas reservas ativas por usuário/dia do campus; canceladas liberam limites.
RN-07 / RN-09	Segundo cancelamento é 409; recurso ausente é 404.
RN-10 / RN-11	Listagem cronológica por usuário; transação conserva invariantes concorrentes.

Arquitetura e fluxo de criação



HTTP valida o formato; serviço inicia transação, consulta sala/limite/conflitos e grava apenas se todas as regras passam. Exceção reverte a gravação. O domínio independe de FastAPI e SQL.

Instalação e execução

```
docker compose up --build api
# Swagger: http://localhost:8000/docs
docker compose --profile test run --rm --build tests
# Local: Python 3.12 + python -m pip install -e ".[dev]"
python -m ruff check .
python -m pytest
```

Docker usa imagem Python 3.12 fixada por digest e volume SQLite. Autenticação, interface web e implantação entre hosts ficam fora do escopo original. user_id não prova identidade; a API é um demonstrador acadêmico local, sem homologação para dados reais de produção.

2. Testes automatizados e execução

Ambiente	Passagem	Cobertura	Comprovação
Windows / Python 3.14.7	55/55 (100%)	99,66%	local.log; exit 0
Docker / Python 3.12.15	55/55 (100%)	99,69%	docker.log; exit 0
GitHub Actions / Python 3.12.15	55/55 (100%)	99,69%	workflow; quality aprovado

Passagem mede testes aprovados sobre coletados; cobertura mede linhas exercitadas. A exigência de 100% de passagem está demonstrada. O gate de cobertura é 90%.

Log real do ambiente padronizado

```
============================= test session starts ==============================
platform linux -- Python 3.12.15, pytest-8.4.1, pluggy-1.6.0
rootdir: /app
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.15.1, cov-6.2.1
collected 55 items

tests/api/test_reservations_api.py .........                             [ 18%]
tests/integration/test_homologation.py ......................            [ 56%]
tests/integration/test_sqlite_repository.py ...                          [ 61%]
tests/unit/test_reservation_service.py .....................             [100%]

============================== warnings summary ================================
../usr/local/lib/python3.12/site-packages/starlette/testclient.py:51
  /usr/local/lib/python3.12/site-packages/starlette/testclient.py:51: DeprecationWarning: The
  anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
------------------ generated xml file: /app/test-results.xml -------------------
================================ tests coverage ================================
_______________ coverage: platform linux, python 3.12.15-final-0 _______________

Name                              Stmts   Miss  Cover   Missing
---------------------------------------------------------------
campusflow/__init__.py                0      0   100%
campusflow/api.py                    57      0   100%
campusflow/domain.py                 20      0   100%
campusflow/errors.py                 20      0   100%
campusflow/repository.py             40      1    98%   100
campusflow/schemas.py                31      0   100%
campusflow/service.py                58      0   100%
campusflow/sqlite_repository.py      89      0   100%
campusflow/time_policy.py             4      0   100%
---------------------------------------------------------------
TOTAL                               319      1    99%
Coverage XML written to file coverage.xml
Required test coverage of 90% reached. Total coverage: 99.69%
======================== 55 passed, 1 warning in 1.63s =========================
```

CI e demonstração HTTP

Execução verificada do commit f863ddb. Artefatos contêm stdout, coverage.xml e test-results.xml (JUnit). Logs completos ficam versionados em docs/evidence/entrega2/.

```
quality Lint  2026-10-14T13:29:28.8811351Z All checks passed!
quality Tests and coverage  2026-10-14T13:29:30.9676544Z platform linux -- Python 3.12.15,
  pytest-8.4.1, pluggy-1.6.0
quality Tests and coverage  2026-10-14T13:29:30.9679208Z collected 55 items
quality Tests and coverage  2026-10-14T13:29:31.6481657Z _________________ coverage: platform linux,
  python 3.12.15-final-0 _________________
quality Tests and coverage  2026-10-14T13:29:31.6488507Z Required test coverage of 90% reached. Total
  coverage: 99.69%
quality Tests and coverage  2026-10-14T13:29:31.6488991Z ======================== 55 passed, 1 warning
  in 0.78s ==========================
```

Fluxo funcional pelo HTTP e reinício

```
GET /health -> HTTP 200
GET /rooms -> HTTP 200
POST /reservations -> HTTP 201
GET /reservations/7ae26f29-87cb-44c5-9629-eef6eae4f8fd -> HTTP 200
POST /reservations -> HTTP 409
GET /reservations?user_id=demo-2dde14c80f324330941f277ddcd72742 -> HTTP 200
DELETE /reservations/7ae26f29-87cb-44c5-9629-eef6eae4f8fd -> HTTP 200
DELETE /reservations/7ae26f29-87cb-44c5-9629-eef6eae4f8fd -> HTTP 409
RESERVATION_ID=7ae26f29-87cb-44c5-9629-eef6eae4f8fd
DEMO_OK: criacao, consulta, conflito, filtro e cancelamento validados

GET /reservations/7ae26f29-87cb-44c5-9629-eef6eae4f8fd -> HTTP 200
PERSISTENCE_OK: estado preservado apos recriacao do conteiner
```

Os logs vieram de execução da aplicação em um projeto Compose de demonstração com dados fictícios. O contêiner foi recriado e o registro continuou cancelado. A instância foi encerrada, preservando o volume. Avisos de depreciação estão mantidos nos logs e não representam falhas.

3. Erros lógicos e re-especificação

Este documento registra falhas reais observadas durante o ciclo; limitações conhecidas não são apresentadas como bugs e resultados não foram inventados.

ERRO-01 - RN-03 aplicada apenas na fronteira HTTP

- **Origem:** Issue #13.

- **Comportamento esperado:** o serviço deve rejeitar qualquer quantidade não positiva.

- **Comportamento observado:** chamadas diretas ao `ReservationService` aceitavam `0` e `-1`, pois apenas o `Field(ge=1)` do Pydantic protegia a rota HTTP.

- **Causa raiz:** uma regra de negócio foi parcialmente delegada ao adaptador de entrada.

- **Teste que revelou:**

`python -m pytest tests/unit/test_reservation_service.py -k non_positive --no-cov`.

```
collected 20 items / 18 deselected / 2 selected
tests\unit\test_reservation_service.py FF
E   Failed: DID NOT RAISE <class 'campusflow.errors.DomainError'>
2 failed, 18 deselected
```

- **Correção:** validação `attendees <= 0` no serviço e erro estável `invalid_attendee_count`; o schema continua rejeitando entradas inválidas cedo, mas deixou de ser a única proteção.

- **Prevenção:** regressão parametrizada com `0` e `-1`; RN-03 e contrato 422 refinados na versão 0.3.0 da especificação.

ERRO-02 - cobertura não coletada no primeiro contêiner

- **Origem:** refinamento do harness da Entrega 1, consolidado na Entrega 2.

- **Comportamento observado:** os 26 testes passavam no contêiner, mas o comando executável `pytest` não carregava o mesmo contexto de cobertura usado pelo módulo Python e o gate reportava 0%.

- **Causa operacional:** diferença entre o atalho instalado e `python -m pytest` no ambiente da imagem.

- **Correção:** `compose.yaml` passou a executar `python -m pytest`, alinhando Docker, execução local e CI.

- **Evidência posterior:** 26 aprovados e 99,48% no Docker da Entrega 1; o log permanece em `docs/evidence/test-run.md`.

Conclusão do loop SDD

Nos dois casos, a correção não foi “fazer o teste passar” isoladamente. O ciclo foi:

`falha observável -> regra/contrato revisado -> implementação mínima -> regressão -> suíte completa`

A homologação só ocorre quando especificação, código e testes descrevem o mesmo comportamento.

Homologação final - 14/10/2026 (Issue #25)

Comando antes das correções: `python -m pytest tests/integration/test_homologation.py --no-cov -q`. Resultado real: **12 failed, 1 passed**. O commit `7808618` preserva os testes anteriores à correção.

Falha	Evidência inicial	Causa	Correção e prevenção
ERRO-03: tipo de participantes	bool/float aceitos; string causa TypeError; HTTP responde 201	coerção HTTP e checagem somente de sinal	inteiro estrito no schema e serviço; 9 casos nos dois adaptadores/HTTP
ERRO-04: ordenação	SQLite retorna 08:00-03 antes de 10:00Z	ORDER BY de texto ISO	converter para datetime e ordenar instante/ID; regressão nos dois adaptadores
ERRO-05: limite diário	terceira reserva aceita em UTC-03 ao escrever duas datas como dia seguinte em Z	`.date()` no offset de cada registro	calendário UTC-03 comum; regressão nos dois adaptadores
ERRO-06: janela de corrida	inspeção mostra checagens e gravação em conexões distintas	ausência de unidade de trabalho	transaction na porta; tests concorrentes de criação, limite, cancelamento e instâncias distintas

ERRO-06 foi identificado estruturalmente; não se afirma que um incidente em produção foi observado. Os testes concorrentes confirmam o estado final, sem depender de qual thread vence. Testes de rollback confirmam que uma falha não deixa gravação parcial. A suíte final mantém todos os testes anteriores.

4. Defesa das decisões de arquitetura

ADR	Decisão / benefício	Trade-off
001	Domínio isolado e porta substituível	mais componentes, menos acoplamento
002	FastAPI/Pydantic, Pytest/Ruff e gate 90%	dependências e manutenção da suíte
003	SQLite local + volume; memória para testes	arquivo local não é banco distribuído
004	UTC-03, tipos estritos e unidade de trabalho	serializa escritores e fixa calendário institucional

- Estado: aceita para homologação da Entrega 2

- Data: 14 de outubro de 2026

- Issue: #25; regras RN-03, RN-04, RN-06, RN-07, RN-10, RN-11

Contexto e alternativas

A comparação de datas locais de cada cliente permitia mais de duas reservas no mesmo dia do campus. A ordenação textual ISO em SQLite não coincide com a ordem dos instantes quando há offsets distintos. Verificações separadas da gravação também deixam uma janela de corrida, mesmo numa única API com múltiplas threads. Apenas exigir testes sequenciais ou limitar a um worker não elimina essa janela.

Decisão

Usar UTC-03 fixo como calendário do campus, preservar o offset de entrada e ordenar datetime pelo instante real. Exigir tipo inteiro estrito para participantes na fronteira HTTP e no serviço. Adicionar `transaction()` à porta: RLock em memória e `BEGIN IMMEDIATE` em SQLite. Operações SQL do caso de uso reutilizam a mesma conexão por contexto; exceção reverte a transação e libera a conexão.

Consequências e trade-offs

- O domínio continua sem SQL e sem FastAPI; o serviço decide regras dentro da unidade de trabalho.

- A ordem dos clientes não determina o limite diário. A política de calendário fica explícita.

- SQLite serializa escritores de instâncias distintas que apontem ao mesmo arquivo; o timeout de conexão é 5 segundos. Instalação distribuída entre hosts permanece fora do escopo.

- Serialização reduz throughput, mas conserva invariantes no volume esperado do demonstrador.

- A política fixa não depende de base de timezone nem de relógio real; mudança do fuso institucional requer refinamento. Datas de teste são constantes.

- PostgreSQL e bloqueios distribuídos foram considerados e adiados por custo operacional. Não é necessário um banco externo para serializar escritores locais com SQLite.

5. Governança e histórico iterativo

Equipe e adaptação autorizada

Felipe Amorim Monteiro, RA 22452139, é o único integrante. `MorimFr` é a conta principal e `SrMorim` é uma segunda conta da mesma pessoa. Em 14/10/2026, Felipe informou que o professor autorizou essa adaptação para o trabalho individual. A autorização é relatada pelo aluno; não há documento do professor anexado a este repositório.

As aprovações nas duas contas representam **autorrevisão humana**, não revisão independente entre pares. O autor continua responsável por ler o diff, validar critérios e homologar as saídas da IA. O relatório final explicita a adaptação em vez de afirmar que existe outro integrante ou colaborador.

Alguns documentos históricos da Entrega 1 usaram a expressão "colaborador externo" antes desse esclarecimento. A caracterização correta para esta entrega é a adaptação individual descrita acima; os artefatos anteriores foram preservados como histórico, não como comprovação de um segundo membro.

Fluxo de branches

- `feature/*` e `fix/*`: tarefas vinculadas a Issues.

- `develop`: integração da sprint via PR, exemplificada pelo PR #20.

- `release/*`: consolidação da árvore validada em develop e promoção via PR para main.

- `main`: versão acadêmica publicada, sempre promovida por PR.

A branch `release/entrega-intermediaria` e o PR #24 foram necessários devido à diferença de ancestralidade de squashes prévios. Merge commits foram habilitados em 14/10 para futuros merges de branches duradouras; squash continua disponível para tarefas curtas. Não há necessidade de reescrever o histórico já publicado nem fazer push direto em main/develop.

Proteções verificáveis

Em 14/10/2026, a auditoria encontrou main/develop sem proteção e sem rulesets ativos. As proteções clássicas foram reaplicadas via API, com configuração versionada em `.github/entrega2-protection.json`:

- PR obrigatório, uma aprovação, descarte de aprovação após novo commit;

- aprovação do último push revisável, por conta diferente da conta que fez o push;

- check `quality`, branch atualizada e resolução de conversas;

- administradores sujeitos às regras; force push e exclusão desabilitados.

Esses controles conferem contas e checks; não verificam independência entre pessoas. Essa diferença está coberta pela adaptação individual informada pelo aluno.

Sprint, critérios e revisão

O marco Sprint 2 contém Issues #13-#19, #21 e #25, atribuídas a MorimFr. Cada tarefa tem critérios de aceitação e referência a requisitos. Fechar tarefas concluídas em develop com evidência do PR correspondente; reservar o aceite da publicação/PDF para a promoção final. Não criar comentários ou aprovações que não tenham ocorrido.

Antes de aprovar, conferir:

1. regra e contrato HTTP atendidos, inclusive erros e bordas;
2. regressão, changelog e ADR para comportamento/arquitetura alterados;
3. nenhum segredo ou banco real no diff; dados de teste fictícios;
4. Ruff, Pytest e Docker aprovados; CI do commit atual verde;
5. leitura humana de transações, rollback, fuso e restrições da solução.

Commits usam `tipo(escopo): resumo`. Logs e referências ficam em `docs/evidence/entrega2/`.

PR	Evidência e função
#20	Feature branch -> develop: SQLite, consultas, testes e documentação; aprovado e mergeado.
#22	Sincronização da Entrega 1; aprovado e mergeado.
#24	Release -> main: homologação; aprovação final pendente no snapshot.

A consulta de status está preservada em github.json. Issues de implementação encerradas referenciam PRs concluídos; #19/#25 acompanham a publicação/PDF. O histórico inclui teste antes da correção, implementação, documentação e evidências.

6. Comparação e análise ético-técnica de IA

Data de atualização: 14/10/2026. Projeto: CampusFlow. Autor responsável: Felipe Amorim Monteiro.

Método e alcance

Codex foi o agente utilizado na implementação e homologação. Claude Code, Cursor e Antigravity foram avaliados documentalmente por fontes oficiais. Não foram executados benchmarks com essas três ferramentas; não há notas de velocidade, custo ou qualidade experimental. O requisito admite ferramentas avaliadas/utilizadas: a matriz compara adequação ao SDD e explicita o tipo de evidência.

Ferramenta	Ponto forte documentado	Limitação/controle relevante	Impacto e evidência
Codex	contexto AGENTS.md, edição e execução de comandos/testes	limites de acesso dependem da configuração, não só do prompt	uso real: implementação e 21 novos casos; geração anterior tinha erros revelados pelo harness
Claude Code	fluxo de terminal com permissões e revisão das ações	permissões amplas/MCP aumentam exposição; mudanças devem ser inspecionadas	avaliação documental: candidato para SDD reproduzível, sem medida de qualidade no CampusFlow
Cursor	editor e agentes integrados; política de Privacy Mode	contexto pode trafegar aos serviços; modo privado não substitui minimização de dados	avaliação documental: facilita leitura de diff e iteração na IDE; sem ensaio prático
Antigravity	política de execução/revisão e sandbox configurável	Always Proceed amplia autonomia e risco; configurar escopo de projeto	avaliação documental: planos e revisão podem apoiar SDD; sem resultado medido

Fontes para as capacidades/controles: Codex AGENTS.md, segurança Codex, Claude Code Security, Cursor Security, Antigravity Agent Settings.

Impacto real na especificação e no código

O Codex auxiliou a decomposição em contratos, porta de repositório e regras numeradas; isso permitiu comparar memória e SQLite sem duplicar os casos de uso. Porém, o código anterior assistido por IA aceitava tipos inválidos, ordenava datas ISO incorretamente e usava calendários diferentes. Uma afirmação de cobertura alta não revelou esses defeitos: em outubro, 13 regressões tiveram 12 falhas antes da correção. A inspeção também apontou uma janela de corrida na gravação.

O benefício mensurável foi expandir a suíte de 34 para 55 casos e documentar regras 0.4.0/ADR-004, com logs vinculados à execução. Não se atribui todo o resultado à ferramenta: o aluno determinou escopo e aceitação, o harness verificou comportamento e a arquitetura delimitou a mudança. Uma comparação experimental futura poderia entregar a mesma Issue a duas ferramentas, com o mesmo commit, prompt, ambiente e orçamento, e medir testes, correções humanas e regressões. Esses dados não existem neste trabalho e não foram simulados.

1. Alucinação e código inseguro ou destrutivo

Agentes podem omitir regras, propor APIs incorretas, instalar dependências desnecessárias ou executar comandos que apaguem/publiquem recursos. Também podem seguir instruções maliciosas em arquivos ou resultados externos (prompt injection). Documentação de segurança oferece permissões e isolamento; existência da opção não prova que ela foi ativada na sessão.

Neste projeto, o controle comprovável é AGENTS.md, Issues, diff versionado, testes, transações e gates do CI/PR. A sessão teve permissões locais amplas; não afirmamos isolamento efetivo de filesystem/rede. Para reutilização, recomenda-se workspace isolado, permissões mínimas e aprovação de ações externas. Instruções textuais ajudam o agente, mas não substituem limites técnicos. Testes concorrentes e de rollback validam invariantes que testes de sucesso sequencial omitiam.

2. Dados, privacidade e confidencialidade

Enviar código/contexto/logs a um modelo pode expor segredos, dados pessoais e informações confidenciais. MCPs acrescentam terceiros e permissões próprios. No Cursor, Privacy Mode reduz uso para treinamento segundo o fornecedor; isso não equivale a proibir qualquer processamento externo. No projeto, identificadores de teste são fictícios; `.env` e banco local ficam fora do Git e do contexto de build Docker. Dados de identificação do aluno ficam no relatório por exigência acadêmica.

A LGPD estabelece princípios de finalidade, necessidade e segurança, e prevê medidas para proteger dados pessoais (arts. 6 e 46). Uma futura implantação precisaria definir finalidade/base do tratamento, acesso, retenção e direitos do titular. `user_id` informado pelo cliente não autentica ninguém; proibir dados reais é um limite do demonstrador, não uma implementação de autorização. Fonte: texto oficial da LGPD.

3. Propriedade intelectual e direitos autorais

Contrato do provedor, licença de dependência e proteção autoral são questões distintas. Os termos OpenAI atribuem a saída ao usuário na medida permitida pela lei e alertam para similaridade e responsabilidade de avaliação. Isso não garante originalidade nem remove direitos de terceiros. Fonte: Termos de Uso.

No Brasil, software tem regime previsto pela Lei 9.609/1998 e a Lei 9.610/1998 trata de direitos autorais. Não presumimos que uma cláusula privada resolva a autoria de toda saída automatizada. Como referência estrangeira, o U.S. Copyright Office estuda a contribuição humana nas obras com IA; seus relatórios não são regra brasileira. Lei de Software, Lei de Direitos Autorais, relatórios USCO.

O aluno deve revisar licenças e procedência, preservar avisos, evitar copiar material proprietário sem permissão e registrar contribuição humana em commits/ADRs. MIT do projeto não concede direitos sobre material de terceiros. Antes de uso comercial, uma revisão de licenças/autoria pode ser necessária.

4. Homologação e revisão humana no SDD

A IA propõe; o responsável confirma intenção, contrato e risco. CI verde é condição técnica, não aprovação acadêmica nem prova de ausência de vulnerabilidades. O processo é Issue/regra, regressão, correção, diff humano, harness, PR e aceite. Os testes não devem ser removidos para ocultar falhas.

Felipe informou que o professor autorizou a adaptação individual: MorimFr e SrMorim são contas do mesmo aluno e as aprovações representam autorrevisão humana. Não houve revisão independente entre pares. A autorização é relatada pelo aluno; não há documento docente anexado. A distinção está documentada porque transparência sobre IA e revisão é parte da ética do trabalho.

Riscos residuais

Risco	Mitigação comprovada/proposta	Limite restante
requisito omitido	IDs e regressões para contratos	requisitos humanos ainda podem ser incompletos
corrupção/conflito concorrente	transação local, rollback, testes	múltiplos hosts e carga elevada não homologados
vazamento ou comando destrutivo	minimização de contexto e privilégio mínimo recomendado	sessão local teve permissões amplas
violação de IP	histórico, licenças e revisão de procedência	ausência de prova automática de originalidade
aprovação inadequada	aceite humano e critérios explícitos	autorrevisão não oferece independência entre pares

As fontes oficiais acima foram consultadas em 14/10/2026. Não foram copiadas extensamente; são referências para a comparação e discussão crítica contextualizadas ao projeto.

7. Comunicação técnica e experiência

Felipe Amorim Monteiro (RA 22452139) realizou o projeto individualmente com apoio do Codex. Informou autorização docente para usar MorimFr/SrMorim na autorrevisão dos PRs. Não houve outro integrante.

A arquitetura da Entrega 1 separou regras e persistência. Na Entrega 2, SQLite substituiu a memória na execução da API, enquanto fixtures mantiveram testes isolados. O exercício confirmou o valor da porta: contratos e regras permaneceram testáveis sem banco externo.

O maior aprendizado foi que cobertura alta não mede completude das regras. A suíte de 34 casos passava, mas a homologação encontrou tipo não inteiro, ordenação por texto e limite diário por offset. As regressões iniciais falharam 12 vezes. Depois da correção e dos casos concorrentes, são 55 testes. Especificação 0.4.0 e ADR-004 explicitam as decisões que antes estavam implícitas.

A verificação/gravação precisava ser atômica mesmo em uma API local: várias requisições podem executar em threads distintas. BEGIN IMMEDIATE resolveu a concorrência de escritores no mesmo banco, com custo de serialização. Implantação entre hosts e autenticação continuam fora do escopo.

O histórico também mostrou o custo de squashes em branches duradouras: main e develop ficaram com conteúdo equivalente e ancestralidade divergente. Um release PR consolidou a árvore e a decisão de merge foi documentada. Não foi necessário descartar trabalho nem fazer push direto na principal.

IA auxiliou implementação e detecção, mas também havia participado da geração do código com falhas. A homologação humana, a especificação e o harness sustentam o aceite. As outras ferramentas foram avaliadas documentalmente; não se atribuem notas experimentais que não foram medidas.

O relato individual descreve aprendizado, decisões e desafios de organização do trabalho, em vez de inventar experiência de colaboração entre integrantes inexistentes.

8. Rastreabilidade da submissão

Exigência do enunciado	Onde está demonstrada
Repositório público + solução	Link na capa; API, SQLite, execução e fluxo HTTP na seção 1/2.
Nome completo + RA	Capa; equipe individual e adaptação nas seções 5/6.
100% de passagem	55/55 no Docker/local/CI, logs reais e links na seção 2.
Erros e re-especificação	Seção 3; SPEC_CHANGELOG 0.4.0 e regressões preservadas.
README, arquitetura e ADRs	README final, diagrama/fluxo, ADRs 001-004 e seção 4.
Comparação e quatro pilares éticos	Seção 6: método, matriz, limites, dados, IP e revisão humana.
Relato de experiência	Seção 7; desenvolvimento individual explicitado.
PRs e sprints	Seção 5; PRs #20/#22/#24 e Issues no marco Sprint 2.

Passo final de homologação

O conteúdo técnico está validado. O PR #24 requer aprovação humana do commit final e merge para atualizar a main. Após o merge, atualizar github.json e executar o gerador para emitir a versão de submissão. A autorização individual foi relatada pelo aluno.

Integridade e reprodução

Fonte do relatório: scripts/build_intermediate_report.py e documentos versionados. Cada log original conserva o resultado observado. Prints são opcionais porque o enunciado aceita logs; nenhuma imagem simulada foi usada como prova. O PDF foi renderizado e revisado para legibilidade antes da entrega ao aluno.